

Exam Analysis

Average marks for acceptance criteria was less since not all edge cases were considered and written correctly.

Agile Methodology

In SDLC Model - a main issue is what ^{time} each phase takes.

You cannot understand how much is executed if you do not start coding.

In traditional way - we cannot go back to the previous phase.

For example in waterfall - suppose, we were planning I need to add a swing with a tree and keep it in the planning phase.

And later we wanted to change it since the outcome was not good.

- This creates loss of time (time waste).
- And going back to planning phase will create waste of time.

Incremental - piece by piece

Iterative - Adding info / stuff

But in Agile model - we take a small section / module and build upon it.

In Agile we use - Incremental + Iterative.

For example in the mid scenario -

- Authentication
- Book App
- Payment
- Consultation

} There are built piece by piece in Agile model

(Feature by Feature)

In traditional document

For the whole application would have been done first then we build. But in Agile we build piece by piece.

Iterative part - Suppose, First I said in authentication add username, email, password.

- Then, we were asked to add age and gender.

For sensitive applications we use - traditional waterfall model
- For example NASA

4 Agile Manifesto values

- 1) Individuals & interaction $\longleftrightarrow$ Processes and tools
 - 2) Working software $\longleftrightarrow$ comprehensive documentation
 - 3) Customer collaboration $\longleftrightarrow$ contract negotiation
 - 4) Responding to change $\longleftrightarrow$ Following a plan.
- Be ready for the project to evolve.
Maximize output within a time constraint.

Hybrid: Following both tradition and agile model

Agile Model Framework

- There are few -

Scrum
XP
Kanban

→ We will discuss in the class.

XP
Scrum
Kanban
scaled Agile process

→ Many times uses both of these.

- Idea Generation

- Generate User story

User story
↓

→ will present a deliverable in 2-3 weeks.

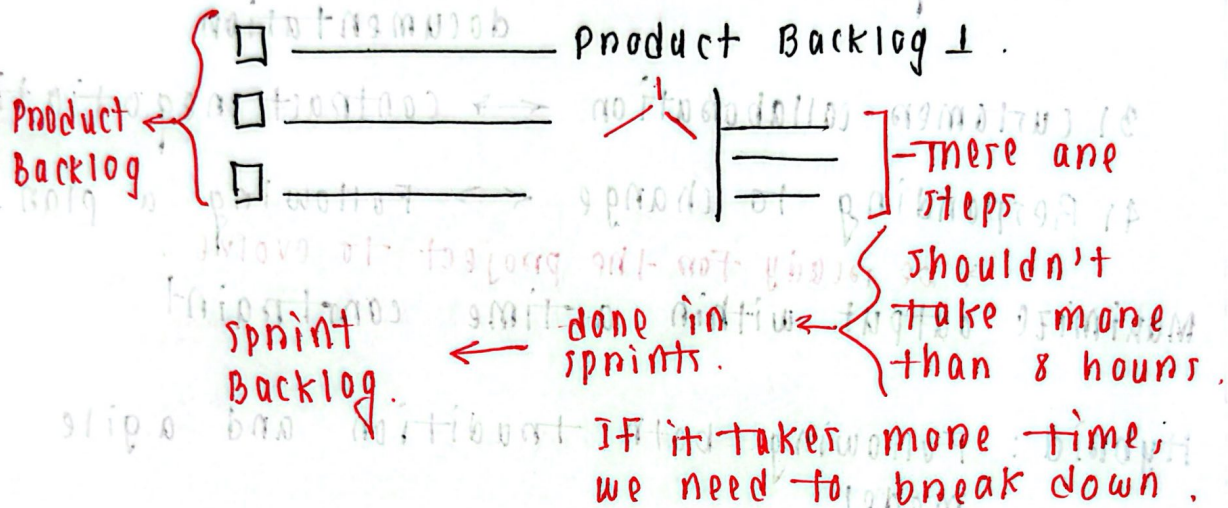
☐ _____
☐ _____
☐ _____

- Product Backlog

IF takes more than 2-3 weeks, it's an epic.

User story

Backwards compatibility.



Definition of Ready :

- checklist a story before put into a sprint.

Definition of Done :

checklist if all the things mentioned in the sprint

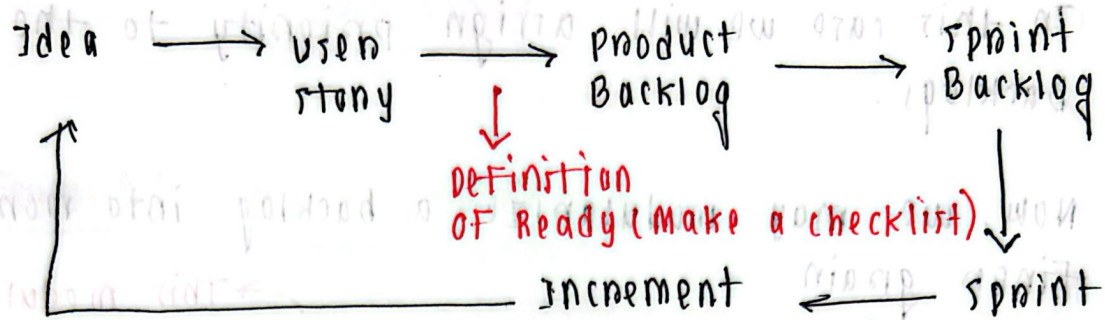
- checklist a story must meet to be is done on not. considered complete.

Team Meeting - sprint meeting - daily at least / max 15 minutes.

Scrum Master - Team lead.

Daily Scrum

IF needed more discussion one-on-one meetings!



- Time constraints for meeting

Product Backlog:

- save user info validate

User Story: Registration

product backlog {
- validate info
- save info

→ Backlog maintains a ordered list.

- Next Iteration (it 2)

- add gender field

- Next Iteration (it 3)

- Remove Pass

Let say we have 10 backlog and I need to do it in 2 weeks, can we do it in that time frame? Depends, right?

creating backlog should not take more than 10% of development time.

In this case we will assign priority to the backlogs.

Now, we may modularize a backlog into more finer grain.

For example: validate info:

- Email
- Age
- username

→ This modularization is sprint backlog.

Moscow way of prioritization

The Moscow method of prioritization

- Must have
- should have
- could have
- won't have

User story - higher level
Sprints, backlog - lower level

Acceptance criteria - Formats

For example user story - I can register using email and password!

Acceptance - Password needs to be 6 characters

Email accepting format.

→ common sense
No technical skill.
Salary more.

→ Dealing with clients.

User story

↳ Not too long.

Format:

As a <role>

I want <goal>

so that <benefit> → From this part, can ask more questions.

Example: As an Account Manager

I want a sales report of my account to be sent to my inbox daily so that I can monitor the sales progress of my customer portfolio.

- User story and functional requirement needs to be actionable and not vague.

3 C's of User story

- card → The Formatted user story.
- conversation
- confirmation.

↳ Basically the Acceptance criteria

- Positive path
- Negative path
- Edge cases
- Exceptions

In Agile Model, there will be changes.

In scrum inside the sprint planning, we normally talk about the user stories.

What Makes a Good User Story?

INVEST

Independent: One user story should not depend on another one, since it will be done by a team and hence they won't ~~work~~ wait to work.

Negotiation: You can break it down or stretch it out. Or we can make it an epic instead.

Value: If that user story is even needed or not.

Estimate: The user story should tell how much time it will need.

For example: In sprint we need to do it in estimated 2-3 weeks.

Small: Small enough to complete it in 2-3 weeks.

Testable: Testable, with clear criteria to verify it meets user needs.

For each User-story, we assign some story points.

User story

→ story points

Assigning points →

1	US1
2	US13
3	US17
5	
8	
13	
:	
21	US10

— Points assigned — expresses the effort needed to complete a user story.

— Here user story 13 needs double the effort of user story 1.

→ Takes lot of effort to complete hence will not be considered as an user story and we'll need to split it.

Planning Poker :

card — Assign story points — 4 team members assign individually

Then they converse with each other, why they assigned each story points and then they conclude the final story points.

That's why we have the conversation part in it.

Alternatives

T-shirts

Xs

S

sizing

US 5

US 7

Affinity Estimates.

1. similar User story
2. Efforts are similar

Epic → Feature.

Epic :

Epic



Feature



User stories



Task 1
Task 2
Task 3

→ Tracked by Product owner / Managers.

→ Manager / Team Leader.

→ Development Team

→ Individual developers

Example :

Epic → Improve the checkout experience

Feature → save payment options.

User story → save address

Task → Home / Apt

- House

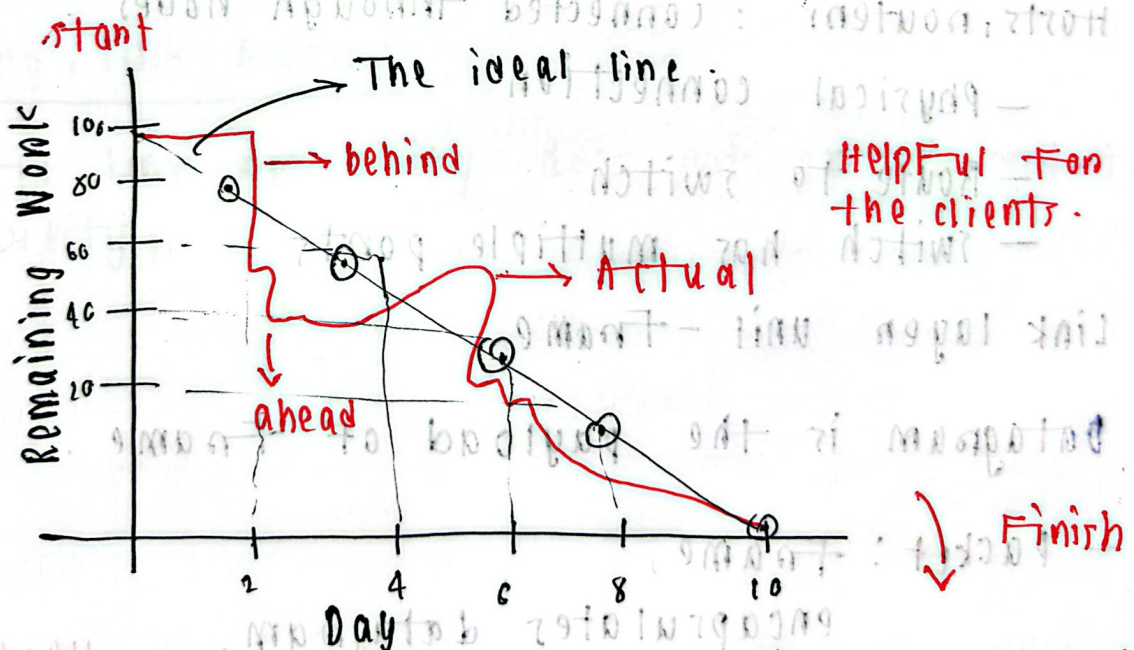
- Road

- Distric

Sprint Burndown chart

Sprint Burndown chart :

This chart is made for each sprint where, it has many user stories.



Will be generated based on checked list.

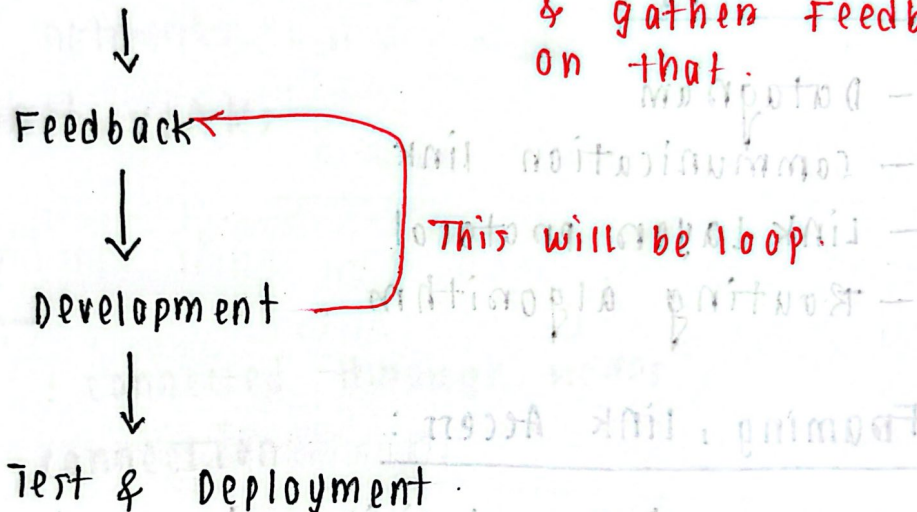
Two different types of development Models :

- RAD ideal type for smaller project
- Prototyping within 90 days.

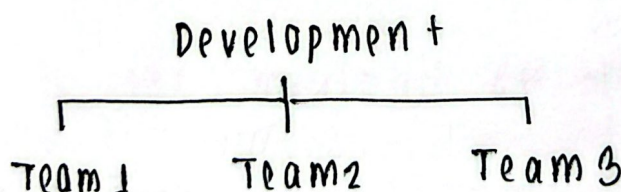
RAD

- Do not have specific format of user requirements.

Phases: Requirement Gathering → create a prototype & gather feedback on that.



Clients are very much involved in this kind of development.



- Always divide the project into smaller modules, so that it can be done, very fast.

- When the possibilities of requirements to change i.e. the requirements are well established and well known, in that case the RAD model is better to be used.
- Timeline / deadline is important (strictly to be maintained).
- You need to have experienced developers
 - ↳ In this case it is easier to implement RAD model.
- High risk and critical system application.
 - ↳ In these cases a detailed SRS documents generally prevails.
- These projects are not ideal for RAD development
- Such application requires waterfall models where there will be no possibility for requirement changes.
- RAD ideal for low risk, smaller application.
- In-house development
 - ↳ In-house department who builds these application.

- Prototype / Mock-up

Prototyping Model:

- Create Prototype on Mock-ups

Start → Requirement Gathering → Quick Design → Building Prototype



stop ← Engineer Product ← Refining Prototype ← Customer Evaluation of Prototype

UI/UX - to create prototypes.

Types of Prototypes:

- Throwaway / Rapid Prototypes:

Discarded after gathering feedback.

- Evolutionary Prototypes

- Incremental Prototypes

- Extreme Prototypes (For web app)

Techniques :

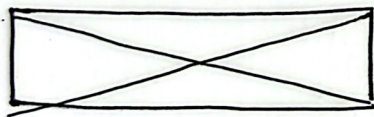
1. Low - Fidelity Prototype

- Storyboarding.
- Show less detailed.
- Main target.
- Story board number
- Button click → Goes to somewhere
assign the number of
story it will go to.

clean detail
↑

There can be low-medium prototyping - Wireframing.
- In wireframing there are notations.

For example (L) stands for logo.



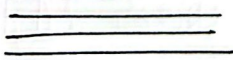
→ Banner / Image



→ Menu.



→ Title



} → More Info.

Footer

sitemap

2. High Fidelity prototype :

- workable prototype but no server related code.

- Interactive .

- Figma button

- Run button .

- scope creep

Final Mid Question Type

- scenario

from that different questions like that in mid

- Theoretical question

scenario given, from that which models ~~are~~
will be better fitted.

↳ RAD or Prototype